

```

1 # Step 1: Secure System Dependency Allocation
2 !pip install -q ultralytics roboflow opencv-python numpy matplotlib
3

```

```

1 # Step 2: Framework Importing & Hardware Optimization Check
2 import os
3 import cv2
4 import torch
5 import numpy as np
6 import matplotlib.pyplot as plt
7 from ultralytics import YOLO
8 from roboflow import Roboflow
9
10 device = "cuda" if torch.cuda.is_available() else "cpu"
11 print(f"[System Initialization] Target compute engine mapped to: {device.upper()}")
12
13 model = YOLO('yolo11n.pt')
14 print("[Framework Status] YOLOv11 nano baseline structural layers successfully initialized.")
15

```

```

[System Initialization] Target compute engine mapped to: CUDA
[Framework Status] YOLOv11 nano baseline structural layers successfully initialized.

```

```

1 # Step 3: Programmatic Real-World Dataset Extraction
2 # Authenticated direct cloud streaming from a verified open universe repository
3 ROBOFLOW_API_KEY = "ciGGavgvfgz1x8UOAPx7"
4 WORKSPACE_NAME = "ntut-fkont"
5 PROJECT_NAME = "dirty-detect"
6 VERSION_NUMBER = 1
7
8 try:
9     rf = Roboflow(api_key=ROBOFLOW_API_KEY)
10     project = rf.workspace(WORKSPACE_NAME).project(PROJECT_NAME)
11     version = project.version(VERSION_NUMBER)
12
13     # Download and extract the real-world dataset directly into the sandbox instance
14     dataset = version.download("yolov11")
15     data_yaml_path = os.path.join(dataset.location, "data.yaml")
16     print(f"[Data Setup] Success! Real-world dataset extracted to cloud storage: {dataset.loc
17 except Exception as e:
18     print(f"[Data Warning] Configuration error, falling back to basic schema. Details: {e}")
19     data_yaml_path = "coco8.yaml"
20

```

```

loading Roboflow workspace...
loading Roboflow project...
[Data Setup] Success! Real-world dataset extracted to cloud storage: /content/Dirty-Detect-1

```

```

1 # Step 4: Full Multi-Epoch Production Training Loop
2 print("[Training Started] Running 50-epoch backpropagation sequence...")
3 training_results = model.train(
4     data=data_yaml_path,
5     epochs=50,
6     imgsz=640,
7     device=device,
8     project="SentinelCleanAIFN_Build",
9     name="production_run",

```

```

10     verbose=True
11 )
12 print("[Training Complete] Custom weights generated. Checkpoint matrices compiled.")
13

```

Epoch	GPU_mem	box_loss	cls_loss	df1_loss	Instances	Size		
43/50	2.7G	1.135	1.277	1.701	4	640: 100%		
	Class	Images	Instances	Box(P	R	mAP50	mAP50-95): 100% -	
	all	69	74	0.365	0.349	0.334	0.191	
Epoch	GPU_mem	box_loss	cls_loss	df1_loss	Instances	Size		
44/50	2.7G	1.119	1.208	1.711	6	640: 100%		
	Class	Images	Instances	Box(P	R	mAP50	mAP50-95): 100% -	
	all	69	74	0.259	0.269	0.327	0.202	
Epoch	GPU_mem	box_loss	cls_loss	df1_loss	Instances	Size		
45/50	2.7G	1.08	1.135	1.63	6	640: 100%		
	Class	Images	Instances	Box(P	R	mAP50	mAP50-95): 100% -	
	all	69	74	0.201	0.393	0.285	0.157	
Epoch	GPU_mem	box_loss	cls_loss	df1_loss	Instances	Size		
46/50	2.7G	1.084	1.155	1.636	4	640: 100%		
	Class	Images	Instances	Box(P	R	mAP50	mAP50-95): 100% -	
	all	69	74	0.265	0.275	0.303	0.179	
Epoch	GPU_mem	box_loss	cls_loss	df1_loss	Instances	Size		
47/50	2.7G	1.051	1.128	1.62	4	640: 100%		
	Class	Images	Instances	Box(P	R	mAP50	mAP50-95): 100% -	
	all	69	74	0.305	0.259	0.307	0.178	
Epoch	GPU_mem	box_loss	cls_loss	df1_loss	Instances	Size		
48/50	2.7G	1.027	1.061	1.596	4	640: 100%		
	Class	Images	Instances	Box(P	R	mAP50	mAP50-95): 100% -	
	all	69	74	0.315	0.302	0.26	0.14	
Epoch	GPU_mem	box_loss	cls_loss	df1_loss	Instances	Size		
49/50	2.7G	1.002	1.049	1.564	4	640: 100%		
	Class	Images	Instances	Box(P	R	mAP50	mAP50-95): 100% -	
	all	69	74	0.325	0.364	0.304	0.175	
Epoch	GPU_mem	box_loss	cls_loss	df1_loss	Instances	Size		
50/50	2.7G	0.9924	1.062	1.573	4	640: 100%		
	Class	Images	Instances	Box(P	R	mAP50	mAP50-95): 100% -	
	all	69	74	0.317	0.316	0.343	0.197	

50 epochs completed in 0.229 hours.

Optimizer stripped from /content/runs/detect/SentinelCleanAIFN_Build/production_run-2/weights/la

Optimizer stripped from /content/runs/detect/SentinelCleanAIFN_Build/production_run-2/weights/be

Validating /content/runs/detect/SentinelCleanAIFN_Build/production_run-2/weights/best.pt...

Ultralytics 8.4.115 🚀 Python-3.12.13 torch-2.11.0+cu128 CUDA:0 (Tesla T4, 14913MiB)

YOLO11n summary (fused): 101 layers, 2,583,322 parameters, 0 gradients, 6.4 GFLOPs

Class	Images	Instances	Box(P	R	mAP50	mAP50-95): 100% -	
all	69	74	0.254	0.269	0.327	0.201	
Brown Stains	4	4	0.444	0.75	0.77	0.598	
Other	2	4	0	0	0.0273	0.00637	
Red Stains	4	4	0.391	0.25	0.34	0.193	
Tea Stains	2	3	0	0	0.0511	0.0141	
Water Stains	56	57	0.691	0.614	0.678	0.329	
White Stains	2	2	0	0	0.0951	0.0666	

Speed: 0.2ms preprocess, 5.4ms inference, 0.0ms loss, 4.2ms postprocess per image

Results saved to /content/runs/detect/SentinelCleanAIFN_Build/production_run-2

[Training Complete] Custom weights generated. Checkpoint matrices compiled.

1 # Step 5: Metrics Extraction & Automated Results Export

2 print("\n--- Final System Analysis Matrix (Honest Performance Reporting) ---")

```

3 evaluation_metrics = model.val(device=device)
4
5 print(f"📊 Achieved Mean Average Precision [mAP@50-95]: {evaluation_metrics.box.map:.4f}")
6 print(f"📊 Achieved Mean Average Precision [mAP@50]: {evaluation_metrics.box.map50:.4f}")
7 print(f"📊 Comprehensive System Recall Margin: {evaluation_metrics.box.mr.mean():.4f}")
8
9 # Create the results directory and systematically save validation charts
10 os.makedirs("../results", exist_ok=True)
11 !cp -r SentinelCleanAIFN_Build/production_run/* ../results/ 2>/dev/null
12 print("[Storage Output] Authentic model training graphs and matrices copied to results/ folder
13

```

--- Final System Analysis Matrix (Honest Performance Reporting) ---

Ultralytics 8.4.115 🚀 Python-3.12.13 torch-2.11.0+cu128 CUDA:0 (Tesla T4, 14913MiB)

YOLO11n summary (fused): 101 layers, 2,583,322 parameters, 0 gradients, 6.4 GFLOPs

val: Fast image access ✅ (ping: 0.0±0.0 ms, read: 1415.3±561.0 MB/s, size: 35.4 KB)

val: Scanning /content/Dirty-Detect-1/valid/labels.cache... 69 images, 0 backgrounds, 0 corrupt: 1

Class	Images	Instances	Box(P	R	mAP50	mAP50-95): 100% —
all	69	74	0.252	0.27	0.327	0.201
Brown Stains	4	4	0.43	0.754	0.77	0.598
Other	2	4	0	0	0.0273	0.00637
Red Stains	4	4	0.391	0.25	0.34	0.193
Tea Stains	2	3	0	0	0.0511	0.0169
Water Stains	56	57	0.692	0.614	0.678	0.328
White Stains	2	2	0	0	0.0945	0.0662

Speed: 13.2ms preprocess, 7.3ms inference, 0.0ms loss, 4.3ms postprocess per image

Results saved to /content/runs/detect/val-2

📊 Achieved Mean Average Precision [mAP@50-95]: 0.2014

📊 Achieved Mean Average Precision [mAP@50]: 0.3268

📊 Comprehensive System Recall Margin: 0.2698

[Storage Output] Authentic model training graphs and matrices copied to results/ folder.

```

1 # --- STEP 6: PRODUCTION LIVE INFERENCE DEMO VIDEO
2 from ultralytics import YOLO
3 import matplotlib.pyplot as plt
4 import os
5 import cv2
6
7 # Load the custom fine-tuned weights
8 trained_model = YOLO('/content/runs/detect/SentinelCleanAIFN_Build/production_run/weights/best
9
10 # Target the pre-separated real validation images folder downloaded in Step 3
11 test_image_dir = '/content/Dirty-Detect-1/valid/images'
12 sample_images = [os.path.join(test_image_dir, f) for f in os.listdir(test_image_dir) if f.ends
13
14 if len(sample_images) > 0:
15     # Select the first real floor stain image from the folder
16     demo_target_path = sample_images[0]
17     print(f"[Demo Core] Processing real floor input imagery: {os.path.basename(demo_target_pat
18
19     # Run instantaneous onboard edge inference
20     demo_results = trained_model(demo_target_path, conf=0.25, verbose=False)
21
22     # Render and display the real vision output box on screen
23     for result in demo_results:
24         annotated_img = result.plot()
25         annotated_img_rgb = cv2.cvtColor(annotated_img, cv2.COLOR_BGR2RGB)
26
27         plt.figure(figsize=(8, 8))
28         plt.imshow(annotated_img_rgb)
29         plt.axis('off')

```

```
30 plt.title(f"SentinelCleanAIFN Active Edge Output: {result.bboxes[0].cls.tolist() if len  
31 plt.show()  
32 else:  
33 print("[Demo Error] Run Step 3 first to extract the test split folder.")  
34
```

[Demo Core] Processing real floor input imagery: images-30-_jpg.rf.03c0f268b61a7a269cae8077263e946

SentinelCleanAIFN Active Edge Output: [4.0]



